

kubernetes

Definition:

- Open Source Container Orchestration tool. by Google
- helps you manage containerized applications on different environments.

What problem does the Kubernetes solve?

The need for kubernetes.

- Trend from Monolith to Microservices.
- Increased usage of containers (so managing huge number of containers with scripts are complex)

Kubernetes tools offer features like

- ① High Availability or No downtime.
- ② Scalability (loads fast and higher response)
- ③ Disaster Recovery (Backup & Restore)

k8s Components:

The fundamental components are

(i) Node & Pod:

- 1) Node in k8s is a simple server which is physical or virtual machine.
- 2) Pod is the basic smallest unit of k8s. Pod

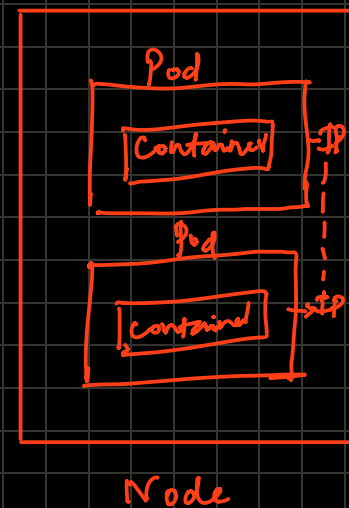
is basically an abstraction over containers. Pod is usually meant to run one container. but we can run multiple containers in one pod.

3.) Pods can communicate with other pods with the IP addresses assigned to them.

↳ private

4.) Pods are ephemeral in nature. New IP addresses when re-creation

If we have Pods with DB, then everytime, new IP addresses have to be assigned & complex thing. So k8s come up with "Service & Ingress"



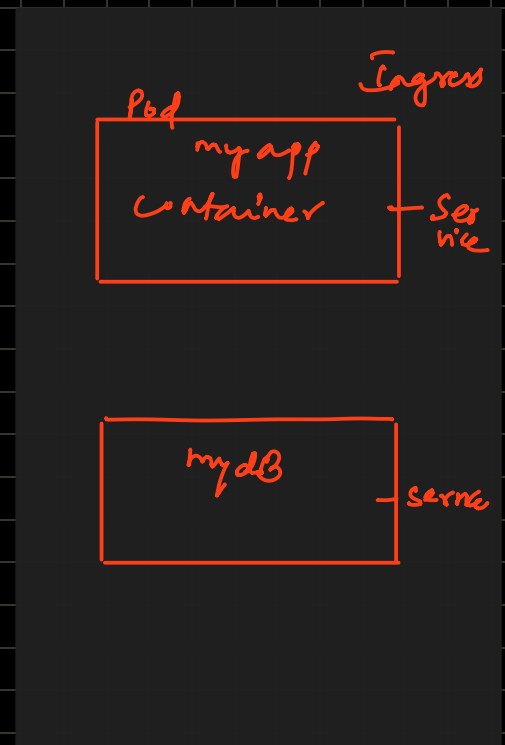
2.) Service & Ingress:

Service:

- permanent IP address
- Lifecycle of Pod and Service are not connected.

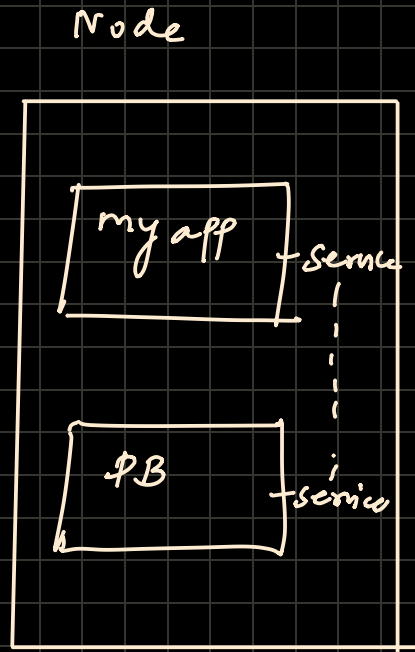
External Service → Example Userfacing

Internal Service → Example DB (not shown to user)



The Ingress will act as a router where it receives requests & forward them to the services.

3.) ConfigMap & Secret:



Let's say my app communicates with MongoDB. The DB URL are usually in build file. The service name of MongoDB changed to mongodb-service then you need to

- rebuild the app
- push it to repo
- pull it into the pods

Tedious work for small DB service name change.

For this purpose, k8s has a Component called ConfigMap:

- external configuration for your app
- Connect this to pods, so that it takes data from the ConfigMap has. In this case, no rebuild, no push & no pull to your pods.

Let's say we have password & username for the DB.
We cannot store it in ConfigMap. (too risky).

For this purpose, k8s has another component called "Secrets"

- used to store secret data (credentials)
- base64 encoded.

"The built in security mechanism is not set by default".

4.) Volumes:

* K8s basically attaches the physical storage on a hard drive to the pod.

* The storage can be local to your pod or remote outside of k8s clusters.

* Think like external (HDD) are plugged into the k8s cluster.

5.) Deployment & Stateful Set.

Let's say we have an app & DB running on a node.
The app gets crashed & down. So we need to create a replica node with access to the service already we have.

So the better way to create an replica is if we already have a "blueprint for my app pods" then each time, we can use this blueprint to create more replica when needed.

This blueprint is called. "**Deployment**". (abstraction of Pods)

Databases (DB) can't be replicated via Deployment. because it has state. For example, if we replicated then from 2 DB pods, the state will be written on storage. Then we need to come up with mechanisms that avoids inconsistencies.

That's why k8s come up with new component called "**StatefulSet**".

This Component specifically meant for apps like databases (Stateful Apps)

So the point is stateful apps can be replicated only thru statefulset not thru deployment.

④ Even database replication thru Stateful is complex. The best practice is to maintain database outside k8s clusters.

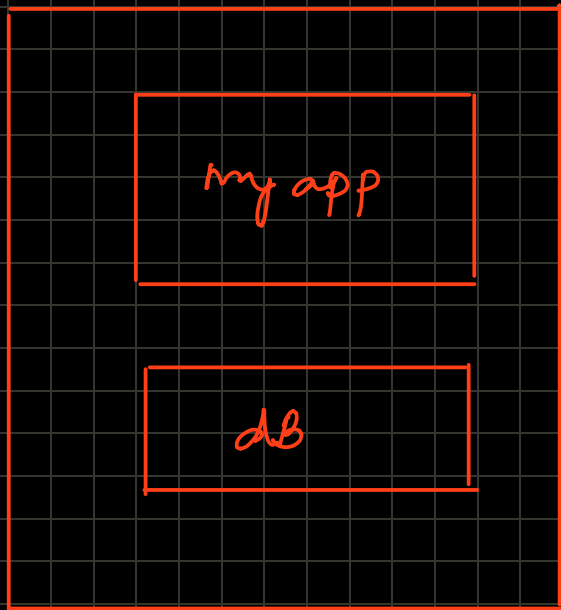
Just have deployment for Stateless Apps.

Kubernetes Architecture

K8s is a master-worker architecture.

Node Processes:

Worker Machine in K8s cluster.



* Each nodes will have multiple application pods

* 3 processes must be installed on each node.

- 1.) container runtime
- 2.) kubelet
- 3.) kubeproxy.

Node 1

Kubelet interacts with both node & containers.

It is responsible for taking the configuration and starting or running a pod and assigning resources from the node to container like CPU, RAM and storage resources.

There can be multiple nodes in clusters and their communication between them via services (acts as load balancer) receives requests & directs to the part of application.

kubeproxy - forwards the requests from services to pods. It is intelligent for forwarding logic and communication done in a way maintaining low overhead.

How do you interact with the cluster?

The answer is Master Node which manages Node processes. But the Master node works in quite different.

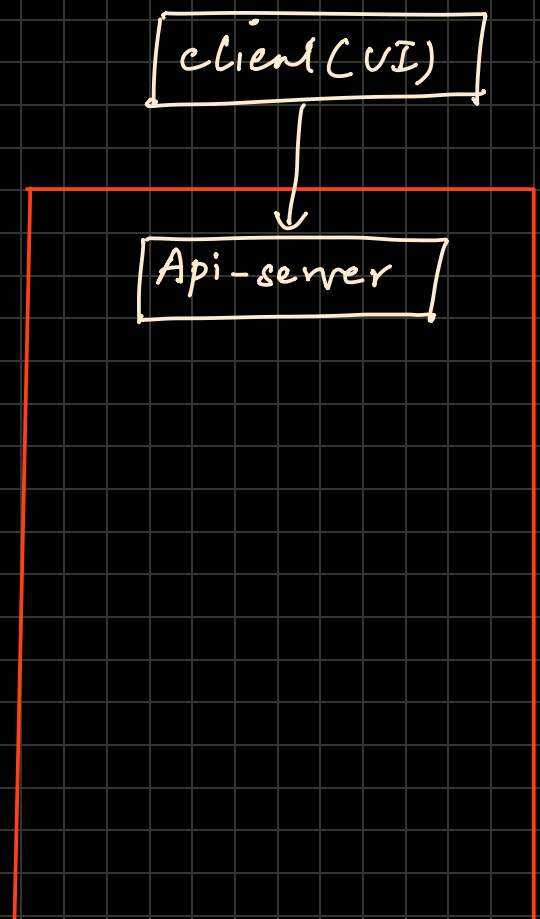
Master Processes

There are 4 processes that manages the cluster state & worker nodes as well.

1.) API Server

→ API server is like a cluster gateway. It gets a initial request of any updates into the cluster or even queries from the cluster.

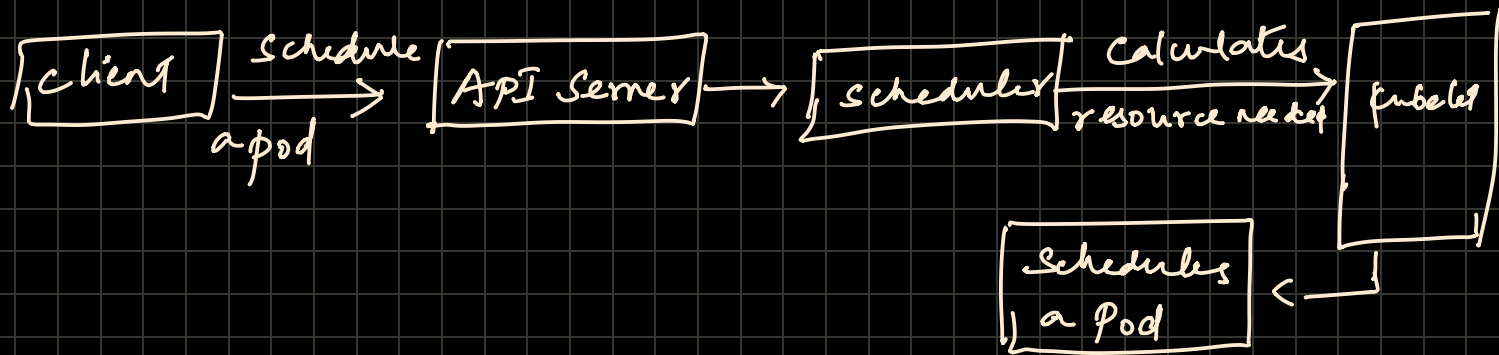
→ Acts as a gatekeeper for authentication.



Master

2.) Scheduler:

Scheduler just decides on which node a pod should be scheduled by checking how much resources a schedule needs and schedules on a node where the usage is less.



3.) Controller Manager:

When pods die on any node, There must be way to detect the node and then reschedule the pods as soon as possible.

So Controller Manager detects state changes like pods die and recovers the cluster state as soon as possible. In the same cycle scheduler decides based on resource calculation which worker node should restart the pods and ask corresponding kubelet to restart the pods.

4.1) etcd:

- * It is The cluster Brain.
- * Cluster changes get stores in key value stores.
- * All other processes in Master Node works based on the data stored in etcd.

Layers of Abstraction:

Deployment manages a replicaset

Replicaset manages all replicas in Pod

Pod is abstraction of container

Kubectl commands:

CRUD Commands:

create deployment: `kubectl create deployment [name]`

edit deployment:

delete deployment:

Status Commands:

`kubectl get nodes/pods/services/replicaset/
deployment`

`kubectl logs [pod-name]`

Get interactive terminal: `kubectl exec -it [pod] --bin/bash`

Use configuration file for CRUD

Apply a configuration file

`kubectl apply -f [file-name]`

`kubectl delete -f [file-name]`

Kubernetes YAML configuration file.

3 parts of yaml file.

1. Metadata

2. Specification (attributes are specific to kind)

3. Status (Desired vs actual)

k8s always checks the status & apply the desired mentioned in yaml (eg: number of replicas)

EtcD holds the current status of any k8's component

Ex:

apiVersion: apps/v1

kind: Deployment

metadata:

name: nginx-deployment

spec:

replicas: 2

selector: ...

template: → This is about the pods. It has separate configuration.

metadata:

labels:

app: nginx

spec:

containers:

- name: nginx

- image: nginx:latest

- ports:

- containerPort: 8080

This is
blueprint
for pods.

Connecting deployment to Pods.

Labels Vs Selectors: Labels will be in metadata where selectors can be seen in spec.

The selector in the spec connects the deployment using the label name of the app.

Ex: selector:

matchLabels:

app: nginx

} This connects the deployment to all pods

Ports in service & pods must match.

Namespace:

→ Organize resources in namespaces

→ virtual cluster in a cluster.

When you create a cluster, kubernetes gives you 4 Namespaces by default.

→ *Kubernetes - dashboard* (only with minikube)

→ *kube-system* (not meant for our use) (It has internal processes)

→ *kube-public* (publicly accessible data like ConfigMap)

- (*kube-node-release*) (heartbeats of a node, determines the availability of node)

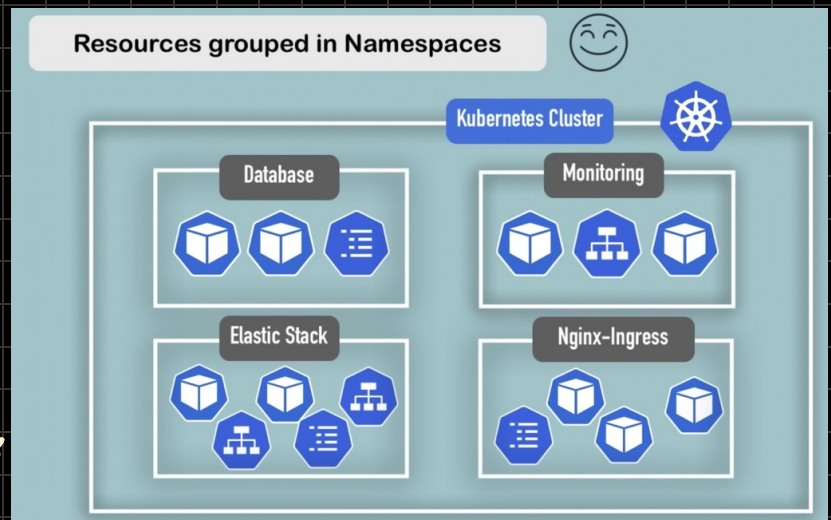
- *default* (all resources created will be here).

So These are the 4 namespaces the kubernetes gives when you create a cluster.

Uses of Namespaces:

Default namespace will be hard to see which deployment/service what does -

Here is the example where you can leverage namespaces



1. Resources grouped in Namespaces.
- 2.) Many Teams but same app, Then create namespace for different teams
- 3.) Resource sharing (Eg Production blue & green)
- 4.) Access & resource limits on namespaces.

INGRESS

The best practice for allowing access to external service is through Ingress by making the external service to internal & allow access thru Ingress.



We need to setup a Ingress Controller pod running inside the cluster. Ingress Controller basically evaluates all the rules in the cluster. This is the entry point of the cluster.

HELM

It is a package manager for Kubernetes.

- To package yaml files & distribute them in public & private repositories.

Example:

Let's say you are adding elastic search logging into your cluster, Then we want Statefulset, ConfigMap, Secret, User Permissions & etc. Getting all these will be tedious process. The solution is helm charts where someone who created these yaml files, gathered it & packaged it once & uploaded to the cloud. (Helm Repository)

Directory structure:

```
mychart/  
  Chart.yaml  
  values.yaml  
  charts/  
  templates/  
  ...
```

Kubernetes Volumes

Need for Volumes:

- 1.) Storage that doesn't depend on Pod life cycle
- 2.) Storage must be available on all nodes
- 3.) storage needs to survive even if cluster crashes.

Persistent Volume:

- a cluster resource that stores data
- created using Yaml file: `kind: PersistentVolume` and in spec how much storage and mention actual physical storage information.
- Persistent Volumes are not namespaced.
- Accessible to whole cluster.

To let the k8s administrator know how much PV needed, its done thru PVC (Persistent Volume Claim). In Pods specification you can mention pvc-claim so that the

Pod → PVC (gets connected)

PVC → PVC (required space)

(*) Claims must be in same namespace

Storage class: SC provisions PV dynamically when PVC claims it.

Levels of Abstraction

- 1.) Pod claims storage via PVC
- 2.) PVC requests storage from SC
- 3.) SC creates PV meet the needs of the claim.

StatefulSet for Stateful Apps

Stateful Applications

Ex: databases,
(basically apps that
stores data)

Stateless Applications

don't keep track of data
each request is completely
new

HTTP



Models

does not depend on previous data. (Just http
(pass time for data updates) request)

↓ update/query

MongoDB

This depends on previous state.

Deployment of Stateless & Stateful Apps.

Replicating stateful apps are more difficult.

- Stateless apps are easy to scale (replicate) where created with random hashes and one service that load balances to any pod. (Replica Pods are identical)
- Stateful Apps (Pods) cannot be deleted/created at same time. & cannot be randomly addressed.
Here the Replica pods are not identical. (Pod Identity)

Pod Identity

It maintains the sticky identity for each pod and created from same configuration but not interchangeable. And each has persistent identifier across any rescheduling.

Why this identity necessary?

